

EXPERIMENT - 12

Title

Develop a Web Application to Manage Student Information using Express and AngularJS.

Aim

To develop a structured full-stack web application using:

- Backend → Express (Node.js)
- Frontend → AngularJS
- MVC Architecture
- REST APIs

This is now a **complete database-integrated Student Management System**.

Final Project Structure

None

student-management/

|

├─ server/

| └─ server.js

```
|   └─ config/
|     └─ db.js
|   └─ models/
|     └─ student.js
|   └─ routes/
|     └─ students.js
|
└─ public/
  └─ index.html
  └─ app.js
  └─ controllers/
    └─ studentController.js
  └─ services/
    └─ studentService.js
  └─ css/
    └─ style.css
  └─
└─ package.json
```

STEP 1: Install Dependencies

Shell

```
npm init -y
```

```
npm install express mongoose body-parser cors
```

BACKEND (Express + MongoDB)

server/config/db.js

JavaScript

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://127.0.0.1:27017/studentfsd_exp12')
  .then(() => console.log("MongoDB Connected Successfully"))
  .catch(err => console.log("MongoDB Connection Error:", err));

module.exports = mongoose;
```

server/models/student.js

JavaScript

```
const mongoose = require('../config/db');

const studentSchema = new mongoose.Schema({

  name: {

    type: String,

    required: true

  },

  course: {

    type: String,

    required: true

  },

  year: {

    type: Number,

    required: true

  }

});

module.exports = mongoose.model('Student', studentSchema);
```

server/routes/students.js

JavaScript

```
const express = require('express');

const router = express.Router();

const Student = require('../models/student');

// GET all students

router.get('/', async (req, res) => {

    const students = await Student.find();

    res.json(students);

});

// ADD student

router.post('/', async (req, res) => {

    const student = new Student(req.body);

    await student.save();

    res.json({ message: "Student Added Successfully" });

});

// UPDATE student

router.put('/:id', async (req, res) => {
```

```
        await Student.findByIdAndUpdate(req.params.id,
req.body);

        res.json({ message: "Student Updated Successfully" });
    });

// DELETE student

router.delete('/:id', async (req, res) => {

    await Student.findByIdAndDelete(req.params.id);

    res.json({ message: "Student Deleted Successfully" });

});

module.exports = router;
```

server/server.js

JavaScript

```
const express = require('express');

const bodyParser = require('body-parser');

const cors = require('cors');

const path = require('path');

require('./config/db');
```

```
const studentRoutes = require('./routes/students');

const app = express();

app.use(cors());

app.use(bodyParser.json());

app.use(express.static(path.join(__dirname, '../public')));

app.use('/api/students', studentRoutes);

app.listen(3000, () => {

  console.log("Server running at http://localhost:3000");

});
```

FRONTEND (AngularJS)

public/index.html

HTML

```
<!DOCTYPE html>

<html ng-app="studentApp">

<head>

    <title>Student Management</title>

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/
angular.min.js"></script>

    <link rel="stylesheet" href="css/style.css">

</head>

<body ng-controller="studentController">

<h2>Student Management System</h2>

<input      type="text"      ng-model="newStudent.name"
placeholder="Name">

<input      type="text"      ng-model="newStudent.course"
placeholder="Course">

<input      type="number"     ng-model="newStudent.year"
placeholder="Year">

<button ng-click="addStudent()">Add</button>
```

```
<br><br>
```

```
<table>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Course</th>
```

```
<th>Year</th>
```

```
<th>Actions</th>
```

```
</tr>
```

```
<tr ng-repeat="student in students">
```

```
<td>{{student.name}}</td>
```

```
<td>{{student.course}}</td>
```

```
<td>{{student.year}}</td>
```

```
<td>
```

```
<button ng-click="editStudent(student)">Edit</button>
```

```
<button  
ng-click="deleteStudent(student._id)">Delete</button>
```

```
</td>
```

```
</tr>

</table>

<script src="app.js"></script>

<script src="services/studentService.js"></script>

<script src="controllers/studentController.js"></script>

</body>

</html>
```

public/app.js

JavaScript

```
angular.module("studentApp", []);
```

public/services/studentService.js

JavaScript

```
angular.module("studentApp")

.service("studentService", function($http) {
```

```
this.getStudents = function() {  
    return $http.get("/api/students");  
};  
  
this.addStudent = function(student) {  
    return $http.post("/api/students", student);  
};  
  
this.updateStudent = function(student) {  
    return $http.put("/api/students/" + student._id,  
student);  
};  
  
this.deleteStudent = function(id) {  
    return $http.delete("/api/students/" + id);  
};  
});
```



public/controllers/studentController.js

JavaScript

```
angular.module("studentApp")

.controller("studentController",          function($scope,
studentService) {

    function loadStudents() {

        studentService.getStudents()

        .then(function(response) {

            $scope.students = response.data;

        });

    }

    loadStudents();

    $scope.addStudent = function() {

        if ($scope.newStudent._id) {

            studentService.updateStudent($scope.newStudent)

            .then(function() {

                loadStudents();

                $scope.newStudent = {};

            });

        }

    }

}
```

```
    } else {  
        studentService.addStudent($scope.newStudent)  
            .then(function() {  
                loadStudents();  
                $scope.newStudent = {};  
            });  
    }  
};
```

```
$scope.editStudent = function(student) {  
    $scope.newStudent = angular.copy(student);  
};
```

```
$scope.deleteStudent = function(id) {  
    studentService.deleteStudent(id)  
        .then(function() {  
            loadStudents();  
        });  
};
```

```
});
```



public/css/style.css

CSS

```
body {  
    font-family: Arial;  
    text-align: center;  
}  
  
input {  
    padding: 6px;  
    margin: 5px;  
}  
  
table {  
    border-collapse: collapse;  
    width: 80%;  
    margin: auto;  
}
```

```
th, td {  
    border: 1px solid black;  
    padding: 8px;  
}
```

```
button {  
    padding: 5px 10px;  
    margin: 2px;  
    cursor: pointer;  
}
```

Execution Steps

Start MongoDB

```
Shell
```

```
mongod
```

Start Server

Shell

```
node server/server.js
```

Open Browser

None

```
http://localhost:3000
```

Features Now Included

Feature	Status
MongoDB Database	✓
Real CRUD	✓
MVC Structure	✓
Update Feature	✓
AngularJS Service Layer	✓

REST API	
----------	---



Final Result

The Student Management System successfully integrates:

- Express Backend
- AngularJS Frontend
- MongoDB Database
- Full CRUD Operations
- Proper MVC Structure

This is now a **complete professional FSD lab implementation**.